CSC317 Computer Graphics

Assignment 4, Part 2: Shaders

Tutorial 6

Adapted from Wenzhi Guo's tutorial notes

October 15, 2026

What are shaders

- Shaders are small programs that run on the GPU to control rendering.
- **Vertex Shader**: Runs once per vertex. Transforms its position (model $\rightarrow$ world $\rightarrow$ camera $\rightarrow$ screen) and outputs transformed positions and other data.
- **Fragment Shader**: Runs once per pixel (or "fragment"). Determines its final color.

Why do we use shaders

- Efficient: The same small program runs on many points in parallel on different GPU cores.
- Flexible: Customizable control over rendering behavior (lighting, color, etc.).
- Save memory: Detail such as color patterns and bumps can be computed on the fly instead of being stored on the GPU.

Implementing software shaders

- Typically shaders are written in OpenGL Shading Language (GLSL)
- A GPU shader sees *one* vertex or pixel at a time.
- Here you write the same math as vectorized NumPy functions that take *whole arrays* of points, so one call shades an entire mesh.

```
# one point at a time (how a GPU shader is written)
for i in range(k):
    colors[i] = shade(P[i], N[i])

# every point at once (how you write it here)
colors = shade(P, N)     # P, N: (k, 3)  ->  colors: (k, 3)
```

Shader Pipeline

What `python main.py --scene <name>` does for every object (`backend/scene.py`):

Geometry (Part 1): `sphere`, `cube`, `catmull_clark`

↓

Bump (optional, spheres only): `bump_position`, `perturbed_normal`

↓

Model matrix: `translate`, `uniform_scale`, `rotate_about_y`
keyframed paths: `catmull_rom_interpolation`

↓

Shade every vertex: `procedural_color`, `planet_color` or `blinn_phong`

↓

Rasterize and display (provided): colors interpolated across faces

NumPy conventions

Shapes:

- A point, normal, direction or RGB color is `(3,)`. Many of them: `(k, 3)`, one row each.
- Most shading functions accept both. Working along `axis=-1` handles both with the same code.
- A per-point scalar is `(k,)`. Use `x[:, None]` to broadcast it against `(k, 3)`.

Matrices:

- `(4, 4)` arrays. `M[i, j]` is row i, column j, as in the math.
- `A @ B` is matrix multiplication; `A * B` is element-wise.
- Homogeneous coordinates: (x, y, z) becomes $(x, y, z, 1)$.

Useful NumPy operations

Operation	NumPy
row-wise dot	`np.sum(a * b, axis=-1)`
row-wise cross	`np.cross(a, b)`
row-wise length	`np.linalg.norm(v, axis=-1, keepdims=True)`
normalize rows	`v / np.linalg.norm(v, axis=-1, keepdims=True)`
element-wise max	`np.maximum(x, 0.0)`
clamp	`np.clip(x, lo, hi)`
linear blend	`a + (b - a) * t`
math functions	`x ** p`, `np.sin(x)`, `np.abs(x)`
choose per point	`np.where(mask[:, None], a, b)`
append ones	`np.hstack([P, np.ones((len(P), 1))])`
`(3,)` $\rightarrow$ `(1, 3)`	`np.atleast_2d(v)`

Part 2 tasks

File in `src/`	Marks	What it does
`model_view_projection.py`	7	Matrices and MVP
`blinn_phong.py`	7	Blinn-Phong shading
`smooth_step.py`	5	Smooth-step curves
`procedural_color.py`	6	Layered-noise marble
`bump_mapping.py`	6	Bump-mapped normals
`catmull_rom_interpolation.py`	6	Keyframe interpolation
`planet_color.py`	6	**Your planet (open-ended)**
	43	

Implement dependencies first:

- `blinn_phong` before `procedural_color`
- `bump_height` → `bump_position` → `tangent` → `perturbed_normal`
- `planet_color` last: it can use everything else

src/model_view_projection.py (7 marks)

Three builders, each returning a `(4, 4)` array:

- `translate(t)`: 4x4 homogeneous matrix that translates a 3D point by "t"
- `uniform_scale(s)`: 4x4 homogeneous matrix that scales all three axes by "s"
- `rotate_about_y(theta)`: 4x4 homogeneous matrix that rotates about the y-axis by "theta" radians

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertex stage itself:
`model_view_projection(model, view, proj, positions)`

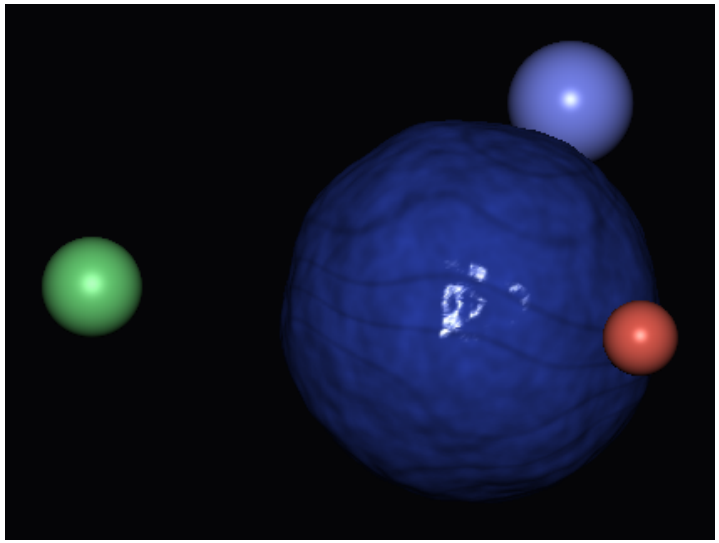
$$\mathbf{p}_{\text{clip}} = \texttt{proj} \cdot \texttt{view} \cdot \texttt{model} \cdot (\mathbf{p}, 1)$$

Takes `(k, 3)` positions and returns `(k, 4)`.

Composing transforms

```
# the moon in main.py --mode shade
def moon_model_matrix(t):
    theta = 2.0 * math.pi * t / 4.0
    return (rotate_about_y(theta)
            @ translate([2.0, 0.0, 0.0])
            @ uniform_scale(0.3))
```

Read right to left: shrink to 0.3, move 2 units along x, then spin about y. The result is an orbit.



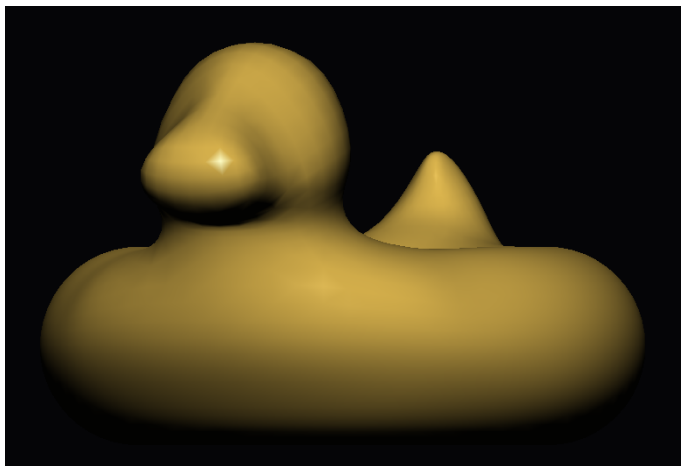
src/blinn_phong.py (7 marks)

`blinn_phong(ka, kd, ks, p, n, v, l)`

$$\mathbf{c} = k_a I_a + k_d \max(\mathbf{l} \cdot \mathbf{n}, 0) + k_s \max(\mathbf{h} \cdot \mathbf{n}, 0)^p$$

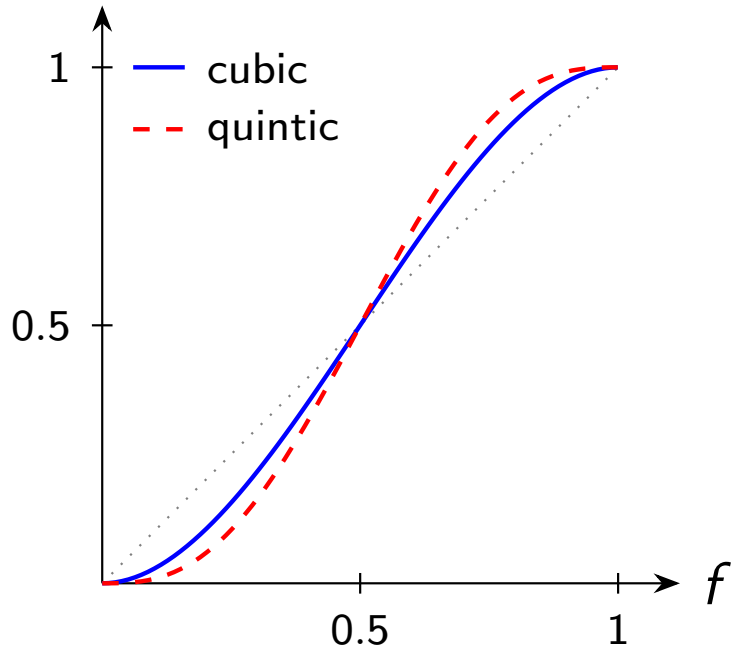
$$\mathbf{h} = \frac{\mathbf{v} + \mathbf{l}}{\|\mathbf{v} + \mathbf{l}\|}, \qquad I_a = 0.1$$

- Normalize **n**, **v**, **l** inside the function.
- Any argument may be `(3,)` or `(k, 3)`. Take dot products over the last axis.

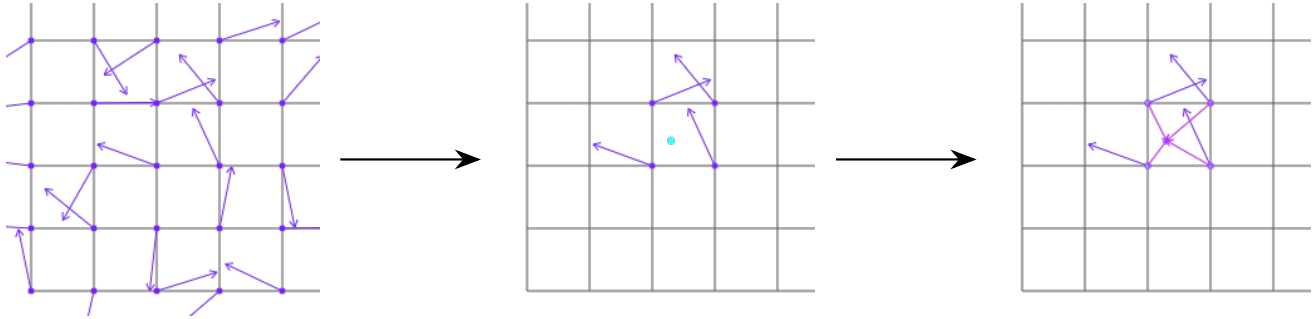


src/smooth_step.py (5 marks)

- Interpolation functions for smooth transitions between two values instead of a linear jump
- `smooth_step(f)` $= 3f^2 - 2f^3$
- `improved_smooth_step(f)` $= 6f^5 - 15f^4 + 10f^3$
- Both map $0 \to 0$ and $1 \to 1$ with zero slope at the ends.
- Must work on scalars and on arrays.



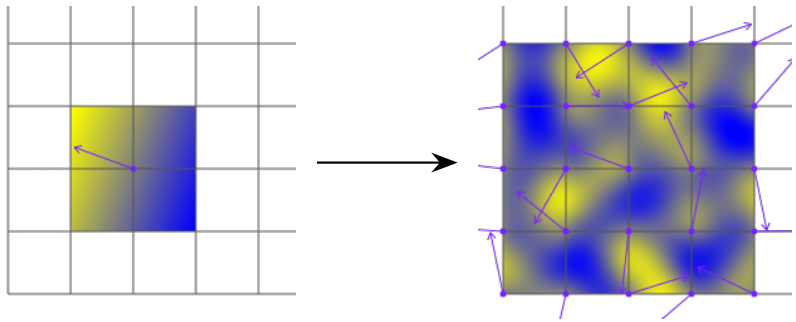
Perlin noise (provided): how it works



1. A random unit gradient at every integer lattice corner

2. Find the cell around the sample point

3. Offset from each corner to the point



4. One corner's term: gradient · offset

5. Blend all corners' terms with smooth-step weights

Credit: `https://www.huttar.net/lars-kathy/graphics/perlin-noise/perlin-noise.html`

Perlin noise (provided): using it

```python
from backend.noise import perlin_noise

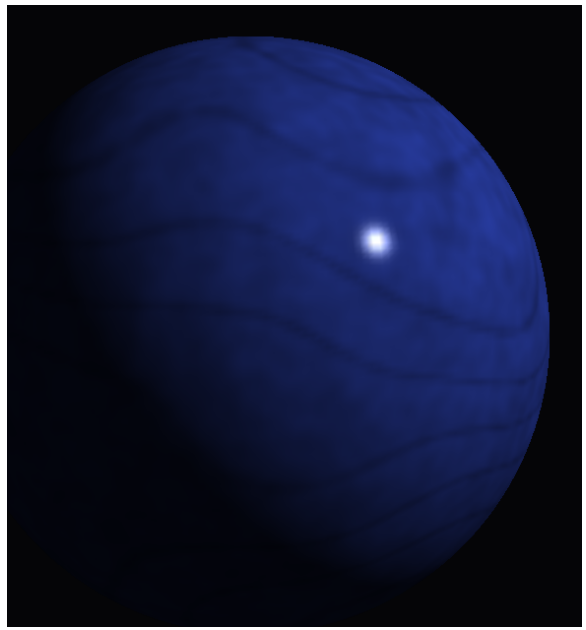
n = perlin_noise(P)     # (k, 3) -> (k,), roughly in [-1, 1]

# double the frequency, halve the amplitude
n = (0.5 * perlin_noise(P)
     + 0.25 * perlin_noise(2.0 * P)
     + 0.125 * perlin_noise(4.0 * P))
```

- Deterministic: the same point always gives the same value, so the surface does not shimmer as the camera moves.
- Scale the input to change the feature size. Shift it (e.g. by `time`) to move the pattern.

src/procedural_color.py (6 marks)

`procedural_color(positions, normals, view, light)`



- Generate the colour of marbled planet with Perlin noise and Blinn-Phong

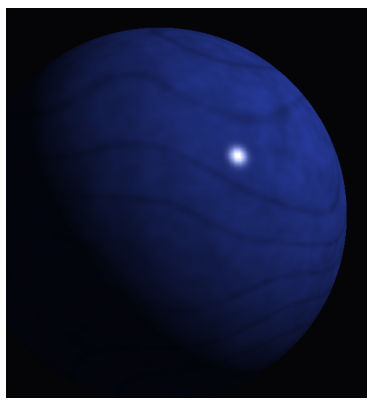
src/bump_mapping.py (6 marks)

- A **bump map** $h(\mathbf{p})$ moves each point along its normal:

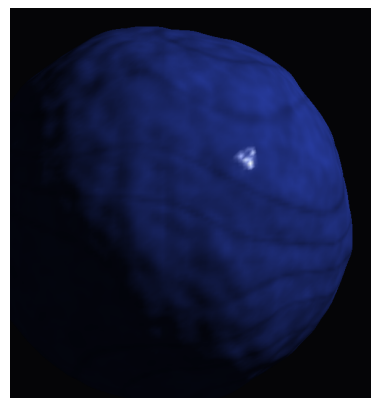
$$\tilde{\mathbf{p}} = \mathbf{p} + h(\mathbf{p})\,\mathbf{n}(\mathbf{p})$$

- The perceived normal $\tilde{\mathbf{n}}$ comes from a finite difference. Step a small ε along two tangent directions $\mathbf{T}$, $\mathbf{B}$, displace, and take a cross product:

$$\tilde{\mathbf{n}} = \mathrm{normalize}\Big(\big(\tilde{\mathbf{p}}(\mathbf{p} + \varepsilon\mathbf{T}) - \tilde{\mathbf{p}}(\mathbf{p})\big) \times \big(\tilde{\mathbf{p}}(\mathbf{p} + \varepsilon\mathbf{B}) - \tilde{\mathbf{p}}(\mathbf{p})\big)\Big)$$



`"bump": false`



`"bump": true`

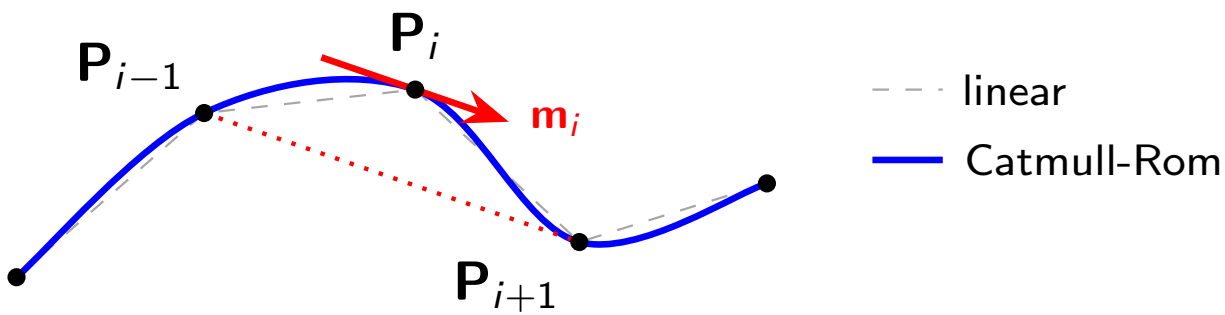
src/bump_mapping.py: four functions

1. `bump_height(s)`: Sums several octaves of Perlin noise into a small signed elevation.
2. `bump_position(s)`: Displace surface points "s" along their (unit-sphere) normal.
3. `tangent(N)` $\rightarrow$ `(T, B)`: For each unit normal N, returns two unit vectors T and B that are perpendicular to N and to each other.
4. `perturbed_normal(points, eps=1e-4)`: the finite difference from the previous slide, built from `tangent` and `bump_position`. Keep the cross product order (`T` step $\times$ `B` step) so the normal points outward. Return unit vectors.

src/catmull_rom_interpolation.py (6 marks)

`catmull_rom_interpolation(keyframes, t)`: `keyframes` is a list of `(time, vec3)` pairs sorted by time. Returns the 3-vector at time `t`.

- Linear interpolation turns sharply at every keyframe.
- Catmull-Rom: one cubic Hermite curve per segment. The tangent at each key is the centered difference of its neighbors, $\mathbf{m}_i = \tau\,(\mathbf{P}_{i+1} - \mathbf{P}_{i-1})$ with $\tau = 0.5$.
- Both segments that meet at a key use the same tangent, so the velocity is continuous (C^1).



src/catmull_rom_interpolation.py: the algorithm

1. **Clamp**: before the first key, return the first value; at or after the last key, return the last value.
2. **Bracket**: *time* k_1, $k_2 = k_1 + 1$. Let $s \in [0, 1]$ be the normalized parameter between them.
3. **Tangents** ($\tau = 0.5$):

$$\mathbf{D}_1 = \tau\,(\mathbf{P}_{k_2} - \mathbf{P}_{k_1 - 1}), \qquad \mathbf{D}_2 = \tau\,(\mathbf{P}_{k_2 + 1} - \mathbf{P}_{k_1})$$

   A missing neighbor at either end is replaced by the endpoint itself, giving $\tau\,(\mathbf{P}_{k_2} - \mathbf{P}_{k_1})$.

4. **Hermite cubic**:

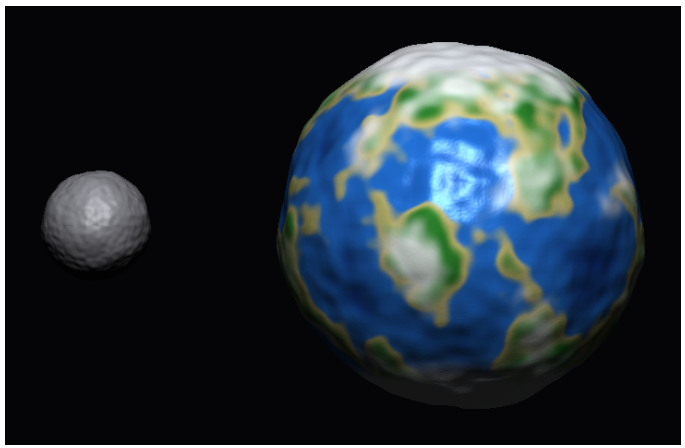
$$h_{00} = 2s^3 - 3s^2 + 1 \quad h_{10} = s^3 - 2s^2 + s$$
$$h_{01} = -2s^3 + 3s^2 \quad h_{11} = s^3 - s^2$$

$$\mathbf{p}(s) = h_{00}\,\mathbf{P}_{k_1} + h_{10}\,\mathbf{D}_1 + h_{01}\,\mathbf{P}_{k_2} + h_{11}\,\mathbf{D}_2$$

src/planet_color.py (6 marks)

```
planet_color(positions, normals, view, light, time=0.0)
```

- Not autograded.
- Use any function from this assignment:
  - `bump_height` as **elevation**: pick a sea level
  - `perturbed_normal` for terrain lighting
  - per-point `(k, 3)` colors in `blinn_phong`
  - `time` for drifting clouds
- Sprinkle noise on anything.



`--scene planet-and-moon`

Useful Resources

- *Fundamentals of Computer Graphics* (4th ed.): review Ch. 6, 7 and §8.1–8.2; read §11.4–11.5 and Ch. 17
- NumPy broadcasting
- Blinn-Phong reflection model (Wikipedia)
- Perlin Noise (Wikipedia)
- Understanding Perlin Noise
- Smoothstep (Wikipedia)
- Bump mapping (Wikipedia)
- Cubic Hermite spline and Catmull-Rom (Wikipedia)